

CSE 4810 : Algorithm Engineering Lab

Lab - 3

MD ATIQUR RAHMAN

Group - SWE A

July 14, 2025

Task 1

Given a square chessboard of $n \times n$ size, the position of the Knight and the position of a target are given. We need to find out the minimum steps a Knight will take to reach the target position.

Hint: BFS

Input:

$n = 6, k = (1, 3), t = (5, 0)$

output: 3

Task 2

Given an undirected graph, the task is to check if the graph contains a cycle or not, using DSU.

Helping Code

BFS

```
def bfs(adj):
    # get number of vertices
    V = len(adj)
    # create an array to store the traversal
    res = []
    s = 0
    # Create a queue for BFS
    from collections import deque
    q = deque()
    # Initially mark all the vertices as not visited
    visited = [False] * V
    # Mark source node as visited and enqueue it
    visited[s] = True
    q.append(s)
    # Iterate over the queue
    while q:
        # Dequeue a vertex from queue and store it
        curr = q.popleft()
        res.append(curr)
        # Get all adjacent vertices of the dequeued
```

```

        # vertex curr If an adjacent has not been
        # visited, mark it visited and enqueue it
        for x in adj[curr]:
            if not visited[x]:
                visited[x] = True
                q.append(x)

    return res

if __name__ == "__main__":
    # create the adjacency list
    # [ [2, 3, 1], [0], [0, 4], [0], [2] ]
    adj = [[1,2], [0,2,3], [0,4], [1,4], [2,3]]
    ans = bfs(adj)
    for i in ans:
        print(i, end=" ")

```

For task 2

```

from collections import defaultdict
class Graph:
    def __init__(self, vertices):
        self.V = vertices # No. of vertices
        self.graph = defaultdict(list) # default dictionary to store graph
    # function to add an edge to graph
    def addEdge(self, u, v):
        self.graph[u].append(v)
    # A utility function to find the subset of an element i
    def find_parent(self, parent, i):
        pass
    # A utility function to do union of two subsets
    def union(self, parent, x, y):
        pass
    def isCyclic(self):
        pass
# Create a graph given in the above diagram
g = Graph(3)
g.addEdge(0, 1)
g.addEdge(1, 2)
g.addEdge(2, 0)

if g.isCyclic():
    print("Graph contains cycle")
else:
    print("Graph does not contain cycle ")

```